EPOCH

INDIVIDUAL WORK ALLOCATION & DELIVERY PLAN

Atul Dhull | 25% of total team workload

Project	EPOCH — Research Integrity & Evidence Workspace
Assigned share	25%
Role	Execution Infrastructure, Sandbox Security & Verification Owner
Architecture baseline	EPOCH_Final_System_Architecture_v2.md (12 November 2026)
Document purpose	Individual execution brief — tasks, deliverables, dependencies and acceptance criteria

Mission

Ensure analyses run in a controlled, isolated, reproducible environment and that failed, malicious or repeated jobs cannot expose data or bypass the integrity rules.

Ownership boundary

Primary owner for supervisor/worker and executor lifecycle, job queue execution, per-job sandbox, resource limits, content-addressed artifact handoff, security/red-team tests and reproducibility environment. Owns operational verification, not verdict mathematics or application workflow policy.

Workload allocation

This assignment represents 25% of the planned team effort. The percentage is a workload share, not a claim that the work is already complete. Scope changes must be agreed by the team and recorded; do not silently transfer unfinished tasks.

Non-negotiable project rules

- Integrity Core must work end-to-end before optional extensions are treated as deliverables.
- Human researchers author hypotheses and plans; the system records, constrains, evaluates and audits the workflow.
- The sealed confirmation split must not be exposed to exploration, ordinary API responses, or AI features.
- Every confirmatory plan is approved and frozen before execution; each confirmatory plan consumes one unit of the lineage budget K.
- Post-hoc work is labelled and descriptive; it must never be promoted to Supported.
- AI is optional, off by default, and never performs statistical calculations or silently edits a human-owned plan.
- All results must be reproducible and verifiable from a signed evidence bundle.

1. Detailed task breakdown

M1 — Execution contract and threat model

- Document the trust boundary: API and supervisor are trusted application services; executor receives only one job's read-only inputs and has no credentials or network.
- Agree with Adithya and Sushma on job payload, result.json schema, exit/error codes, artifact directory and idempotency key.
- Create a threat matrix for malicious CSV/parser inputs, path traversal, resource exhaustion, cross-job contamination, duplicate jobs, partial outputs and container escape.

M2 — Supervisor and job lifecycle

- Implement or configure a supervisor that claims jobs from PostgreSQL using leases/heartbeats or the approved queue library; it must not run user-controlled analysis code.
- Ensure job state and run records commit consistently. Use plan hash/idempotency constraints to prevent duplicate confirmatory jobs and double spending.
- Stage only the job's input files into a dedicated spool directory, read-only to the executor; validate result.json against a strict schema before accepting outputs.
- Handle timeout, crash, retry, cancellation and worker restart. A crash retry must not expose partial output or spend K; once confirmation rows are read and output could reach the user, apply the v2 spend rule.

M3 — One-job executor isolation

- Build the executor image with pinned dependencies and minimal privileges. Run one job per container, then destroy it; never reuse a container for another job.
- Use read-only root filesystem, tmpfs scratch, one job-specific spool subdirectory, no network, no credentials, CPU/memory/wall-clock limits and OMP_NUM_THREADS=1.
- Use gVisor runsc on Linux. For Windows development, document WSL2 + Docker Engine workflow; run gVisor verification in Linux CI or a Linux host. Do not claim gVisor works natively on Windows.
- Ensure no container mounts /var/run/docker.sock. The supervisor may orchestrate containers but no user input runs in it.

M4 — Verification, security and benchmark support

- Write tests proving executor A cannot read executor B's files, cannot access network/secrets, cannot write input files and is destroyed after one job.
- Test malicious CSVs, oversized files, malformed encodings, formula-like column names, path traversal, timeouts and memory exhaustion.
- Verify environment pinning, artifact hashes and deterministic result transfer; support epoch verify/reproduce acceptance tests.
- Support E4 simulation benchmark and scripted p-hacker arm by providing repeatable isolated execution and capturing runtime/resource metrics; do not own statistical interpretation.

2. Required deliverables

Every item below must be produced, committed or linked, and demonstrated. A verbal update alone is not a deliverable.

- Threat model and trust-boundary diagram for API, supervisor, spool and executor.
- Supervisor/job worker implementation with leases, heartbeats, idempotency and failure handling.
- Single-job executor image/configuration with Linux gVisor path and Windows WSL2 development guide.
- Strict result/artifact validation and content-addressed handoff to backend storage.
- Security test suite and red-team report with pass/fail evidence.
- Reproducible environment lock/image digest and runbook for local execution and CI.

3. Definition of done and acceptance criteria

A deliverable is accepted only when the relevant evidence exists (tests, API contract, migration, screenshot plus working route, reproducible command, or review record).

- Each executor processes exactly one job and exits; it cannot see any other job's spool directory.
- No executor has network access, credentials, Docker socket or writable access to input files.
- Timeout/crash retries produce no partial result and do not double-spend budget K.
- Malformed or hostile uploads fail safely with bounded CPU, memory and wall-clock use.
- Every accepted result matches the agreed schema and artifact hashes; tampered artifacts are rejected.
- Linux CI demonstrates the gVisor execution path; Windows setup instructions accurately use WSL2/Linux.

4. Dependencies and handoffs

- Sushma supplies queue tables, run state transitions, database access pattern and storage metadata.
- Adithya supplies engine entry points, deterministic result schema and verification commands.
- Pranav needs job/run status and error codes for loading, failed and completed UI states.
- Team must provision a Linux-capable environment for gVisor verification and agree resource limits.

5. Explicitly out of scope

- Do not implement the statistical verdict evaluator or user-facing investigation CRUD.
- Do not expose Docker socket to the API, supervisor or executor.
- Do not run arbitrary notebooks, shell commands or user scripts in the MVP.
- Do not deploy publicly before the red-team suite passes.

6. Milestones and sequencing

Follow the architecture's M0–M7 build order. The dates are intentionally not invented here; the team should set calendar dates after M0 confirms the dataset, question, capacity and technical feasibility.

Milestone	Your expected contribution
M0 — Decide	Threat model, host/runtime choice, job contract and resource limits.
M1 — Engine/infra spike	Prove one minimal job can execute and return validated result in the chosen Linux isolation setup.
M2 — Backend	Connect PostgreSQL queue, supervisor, per-job executor and storage handoff.
M3 — UI slice	Provide reliable run statuses/errors to the UI and complete an end-to-end run.
M4–M6 — Hardening	Complete red-team suite, backups/restore support, benchmark harness execution and security evidence.

7. Required weekly progress update

- Completed: list merged work and link the PR/commit, migration, test report or document.
- In progress: identify the exact task, current status and next concrete action.
- Blocked: name the dependency, who can unblock it and the decision needed.
- Evidence: provide command output, test result, API response, short recording or reproducible demo.
- Risk/change: state any scope change, security concern, schema change or estimate change before acting on it.

8. Collaboration and handoff checklist

- Agree interface/schema changes with dependent owners before implementation; avoid parallel incompatible assumptions.
- Open a focused pull request with a clear summary, test evidence and known limitations.
- Include setup/run instructions and sample configuration without secrets.
- Never commit credentials, signing keys, dataset secrets or personal/regulated data.
- Demonstrate the feature from a clean checkout or documented environment before marking it complete.

9. Review 1 contribution

For Review 1, present the threat model, executor boundary, one-job lifecycle, gVisor/WSL2 feasibility spike and the exact failure cases the design prevents. If the sandbox is not implemented, show a tested prototype and state what remains.

10. Shared project completion standard

- All code is committed to the agreed repository and the team can run it from documented setup instructions.
- Interfaces, request/response schemas, database migrations and environment variables are documented.
- Unit tests and integration tests are included; no task is considered complete on the basis of a screenshot alone.
- Known limitations, assumptions, risks and deferred features are recorded rather than hidden.
- Changes are reviewed by the relevant module owner and integrated through the shared branch/PR process.

11. Architecture traceability

This work plan is derived from the consolidated v2 architecture. Primary sections relevant to this assignment: §3 Scope tiers; §4 System architecture; §6 Website journey; §7 Integrity Core; §8 Extensions; §9 Data model; §10 Engine interface and test catalogue; §11 AI guardrails; §12 Security/privacy/reliability; §13 Evaluation/benchmark; §14 Build order; §15 Acceptance criteria; §16 Risks; §17 Open decisions; §21 v2 corrections. The individual plan does not override the architecture.

Important status note: the architecture is a proposed design, not evidence of implementation, load testing or security certification. Each owner must report actual implementation status honestly.